



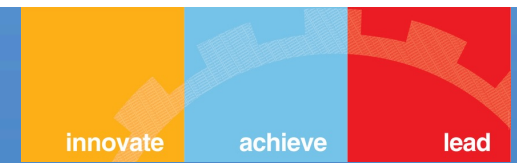
BITS Pilani

Conversational AI

Vector Embeddings and ANN Search algorithm

Dr Bharathi R

Adjunct Professor, CSIS WILP BITS PILANI



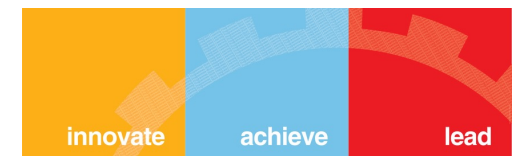
BITS Pilani



AIMLCZG521 , Conversational AI

Lecture No. 2 | Module 1

Disclaimer & Acknowledgement



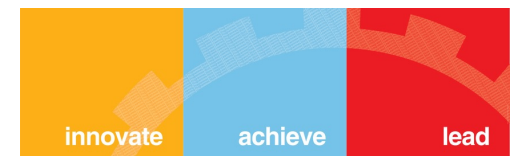
DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, along with special recognition of Prof. Bhagath's Conv-AI-WILP (October 2025) presentation materials, which have been adapted and modified to support the learning objectives of this course.

Outline

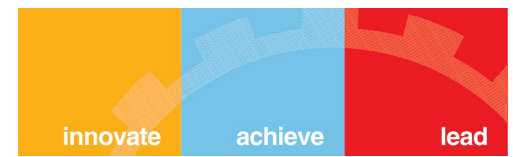


Part 1 : Embeddings

- Encoder models & embedding generation
- Transformer architecture for embeddings
- Vector similarity mathematics
- Embedding model landscape

Part 2: ANN search algorithms

- ANN search algorithms (HNSW, IVF, PQ)
- Index structure mathematics
- Database solutions & tradeoffs
- Cost engineering



Prerequisites Review

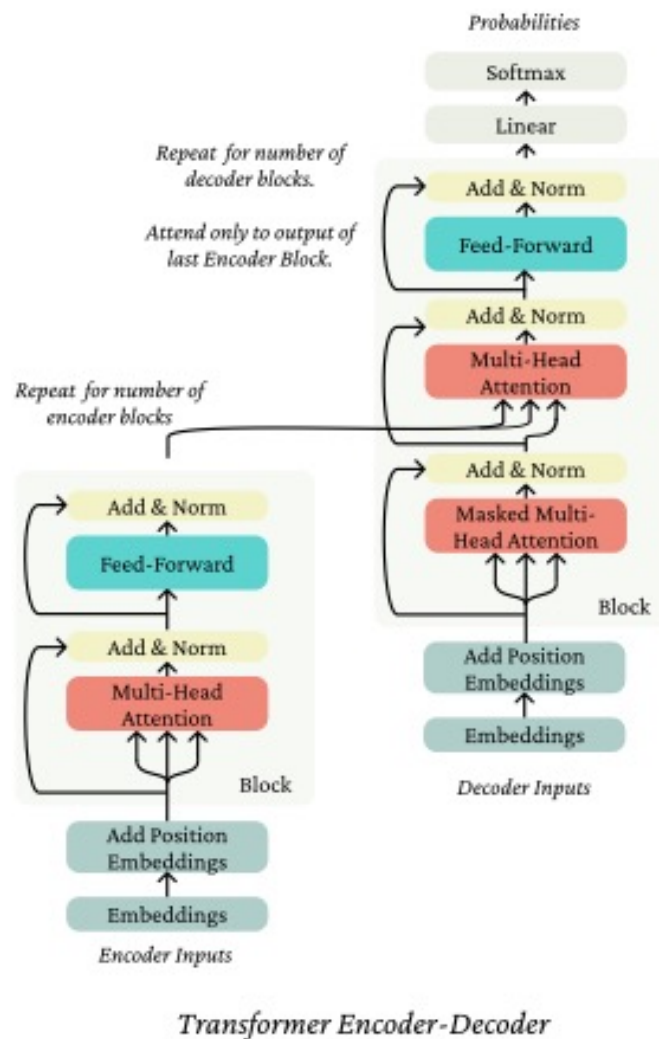
Basic transformer architecture
(self-attention, feedforward layers),
matrix operations.

Learning Objectives



- ✓ **Explain** how transformer encoder models create contextual embeddings through self-attention and pooling strategies
- ✓ **Calculate and compare** vector similarity using cosine similarity, L2 distance, and dot product metrics
- ✓ **Evaluate and select** appropriate ANN indexing strategies (HNSW, IVF, PQ) based on dataset size, memory constraints, and accuracy requirements

Transformer Architecture



Full Transformer Block

Embeddings + Positional Encoding

Token IDs → dense vectors + position info

Multi-Head Self-Attention

Parallel attention over different subspaces

LayerNorm + Residual

Add & normalize after each sub-layer

Feed-Forward Network (FFN)

Two linear layers with ReLU; applied per position

Nx Layers (Encoder/Decoder)

Stack of N identical blocks

Linear + Softmax (Decoder)

Project to vocab size → probability distribution

Self-Attention vs Cross-Attention- Revisit

Sentence: "The bank was closed because the river overflowed its banks."

Self-Attention's Job: When the model reaches the first word 'bank,' it looks at the rest of the sentence. It sees 'river' and 'overflowed.'

The Result: It realizes the first "bank" refers to a **landform** (river bank), not a **Financial Institution**.

Key Point: The "query" and "result" both come from the **same sentence**.

Input (English): "I love Mangoes."

Output (Hindi): "Mujhe Aam pasand hain."

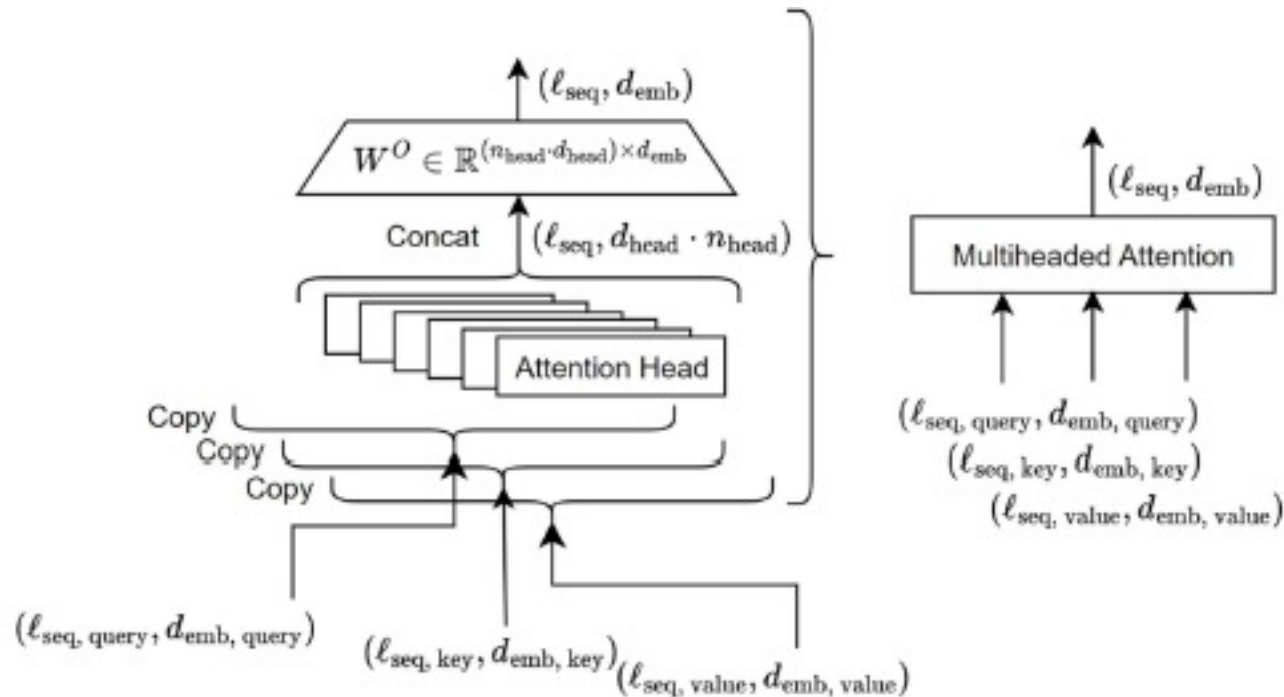
Attention's Job: When generating the Hindi word "Aam," its Attention mechanism looks back at the English sentence.

The Result: It "attends" to the word "**Mangoes**" — the most relevant piece of information for that Hindi word.

Key Point: The "query" comes from the **output (Hindi)**; the "data" from the **input (English)**.

Mechanism	Location	Q (Query) Source	K & V (Key/Value) Source
Self-Attention	Encoder & Decoder	Current Layer Input	Current Layer Input
Cross-Attention	Decoder Only	Decoder (Current state)	Encoder (Final output)

Multi-headed Attention



Core Concept: Instead of performing a single attention function, Multi-Head Attention runs h parallel attention heads, each with different learned projections for Q, K, and V. Outputs are concatenated and projected through W^O to the final embedding dimension.

What are Encoder Models?



Bidirectional Transformers for Understanding

Core Capability: Encoder models can see the **entire input sequence** at once (bidirectional context), making them ideal for:

- Understanding meaning (semantic representation)
- Classification tasks (sentiment, intent)
- Creating embeddings for similarity search

BERT (2018)

Training: Masked Language Modeling (MLM)

"The [MASK] brown fox [MASK] over the lazy dog"
→ Predict: "quick", "jumps"

Key insight: Random tokens replaced with [MASK], model predicts independently. Missing tokens can't be used for generation.

GPT (2018)

Training: Autoregressive (Causal LM)

"<s> A B C D" → Predict each token autoregressively

Key insight: Tokens predicted left-to-right. Can condition on leftward context, so it can be used for generation.

Why Encoders for Embeddings?



The Encoder: The Master of Understanding

Encoder Models, with their **bidirectional attention** (seeing words on BOTH sides), solve the problem of polysemy by creating Contextual Embeddings.

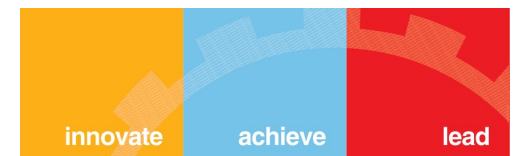
A Contextual Embedding is a vector representation of a word that changes based on the sentence it appears in.

The Mechanism of Contextualization

Because the Encoder's **self-attention mechanism** (which is bidirectional) allows the word "bank" to be influenced by all other words in the sentence, the final vector it produces is a precise, context-specific representation of the word's meaning in that exact moment.

Sentence	Word Being Processed	Encoder's Bidirectional View	Resulting Embedding for "bank"
"river bank"	bank	Looks at "river" (left) and any words that follow (right).	The embedding vector is pulled closer in the vector space to words like "water," "shore," and "geography."
"bank account"	bank	Looks at "account" (right) and any words that precede (left).	The embedding vector is pulled closer in the vector space to words like "money," "finance," and "deposit."

Encoder vs Decoder



Encoder

Input Processing

(e.g., text, tokens)

Bi-directional

Attends to entire sequence
Context from both directions

Self-attention

Parallel processing
Position-aware encoding

Encoders see **full context** → Perfect for understanding, classification, and creating embeddings for similarity search!

Decoder

Output Generation

(probabilities)

Uni-directional

Causal masking
Left-to-right generation

Auto-regressive

Sequential generation
Token-by-token output

Masked Self-attention +

Cross-attention

Attends to encoder output
Prevents future peeking

Decoders Laugh as Loud as Encoders

Borodach, Eli, et al. "Decoders Laugh as Loud as Encoders." *arXiv preprint arXiv:2509.04779* (2025).

0.86

Best Encoder F1
RoBERTa-base

0.85

Fine-tuned GPT-4o F1
Statistically equal

6

Humor Categories
5 types + no-joke

1,392

Dataset Examples
Human-authored jokes

MODEL TYPES COMPARED

ENCODERS

BERT · RoBERTa · DeBERTa ·
XLNet · ALBERT · ModernBERT ·
NeoBERT...

Best: RoBERTa-base F1 = 0.86 (fine-tuned, 17 models tested)

Prior SOTA — still the best encoder; size helps except in RoBERTa & ALBERT

ENCODER-DECODERS (E2D)

BART-large-mnli · Flan-T5-base

Best F1 $\approx$ 0.18 (zero/few-shot only — not fine-tuned)

Flan-T5 few-shot < zero-shot: long prompts overwhelm small E2D models

DECODERS

GPT-4 · GPT-4o · Llama 3.2 ·
Gemma 2 · Qwen2 · Mistral

Fine-tuned GPT-4o F1 = 0.85 $\approx$ RoBERTa ✓ (statistically equal)

Generative models match classifiers — contradicts the encoder-superiority assumption

★ *Core Result: Classifying sentences into 6 categories (5 humor types + no-joke), both fine-tuned Encoders & Decoders perform equally well — overturning the prior assumption that Encoders (RoBERTa) are superior.*

★ *Earlier belief: Encoders are better for understanding tasks (like classification)*
*This paper shows: **Fine-tuned decoders can match encoder performance***

Encoder vs Decoder vs Encoder-Decoder



Encoder-Only

Core Strength : Bidirectional context understanding. Sees entire input simultaneously — ideal for classification and understanding.

Best Use Cases

- Sentiment Analysis – full context for mixed emotion
- Named Entity Recognition (NER) – token-level
- Question Answering – extract answer span
- Text Classification – spam, intent detection
- Semantic Search & Embeddings

•**Popular Models (2025)** : BERT • RoBERTa • DeBERTa • ALBERT • DistilBERT

Decoder-Only

Core Strength : Causal language modeling with autoregressive generation. Predicts next token — ideal for generative tasks.

Best Use Cases

- Text Generation & Completion – creative writing, code
- Conversational AI & Chatbots – ChatGPT, Claude
- Code Synthesis – GitHub Copilot
- In-Context Learning – few-shot prompting
- Instruction Following – reasoning, tool use

•**Popular Models (2025)** : GPT-4 • GPT-3.5 • Claude 3/4 • Llama 3 • Mistral

Encoder-Decoder

Core Strength : Separate encoding and decoding phases with cross-attention. Encoder sees bidirectionally; decoder generates while attending.

Best Use Cases

- Machine Translation – English to French
- Text Summarization – encode doc, decode summary
- Text-to-Text Tasks – paraphrase, style transfer
- Question Answering (Generative)
- Document Generation from Structured Data

•**Popular Models (2025)**: T5 • BART • mT5 • PEGASUS • mBART • FLAN-T5

Comparison

Task Type	Encoder-Only	Decoder-Only	Encoder-Decoder	Reasoning
Sentiment Analysis	✓ Best	○ Possible	✗ Overkill	Needs bidirectional context, no generation required
Text Generation	✗ No	✓ Best	○ Possible	Requires autoregressive generation capability
Machine Translation	✗ No	○ Suboptimal	✓ Best	Cross-attention optimally aligns source-target pairs
Named Entity Recognition	✓ Best	✗ Inefficient	✗ Overkill	Token-level classification needs bidirectional context
Conversational AI	✗ No	✓ Best	○ Outdated	Decoder-only scales better for dialogue and instruction following
Summarization	✗ No	✓ Modern	✓ Traditional	Both work; decoder-only now preferred for unified architecture
Semantic Search	✓ Best	○ Possible	✗ Overkill	Encoder creates optimal dense representations for similarity
Code Generation	✗ No	✓ Best	○ Rare	Sequential generation with context, decoder-only excels

How Encoder Transformers Create Embeddings



6 tokens → Single 768-dimensional vector

Input Text: "Machine learning is fascinating"

Step 1: Tokenization

- [CLS] Machine learning is fascinating [SEP]
- Token IDs: [101, 8394, 4083, 2003, 17117, 102]

Step 2: Token Embeddings

- Each ID → 768-dim vector
- 8394 → [0.23, -0.15, 0.89, ..., 0.42]

Step 3: Position Embeddings

- Position 0 → [0.01, 0.02, ...]
- Position 1 → [0.02, 0.03, ...] (Sum with token embeddings)

Step 4: Transformer Layers (12 layers for BERT-base)

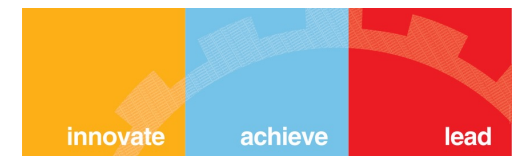
Each layer: Self-Attention → LayerNorm → FFN → LayerNorm

Step 5: Pooling Strategy

- CLS Pooling (most common): $v = h[\text{CLS}]$
- Mean Pooling: $v = (1/n) \sum h_i$
- Max Pooling: $v_j = \max(h_{1j}, h_{2j}, \dots, h_{nj})$

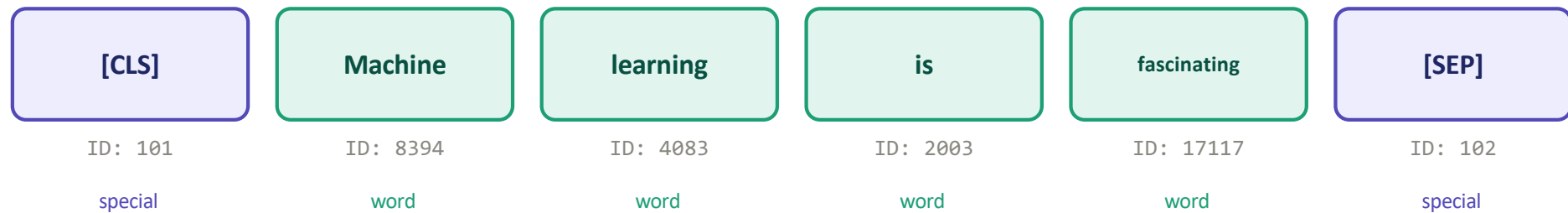
Default Strategy: Most pre-trained models (BERT, RoBERTa, DistilBERT) use **CLS token pooling** by default. However, **mean pooling** often performs better for sentence embeddings (Sentence-BERT, BGE, GTE).

BERT Pipeline



Step 1 — Tokenization

BERT wraps every sentence with special tokens before processing:



Token IDs: **[101, 8394, 4083, 2003, 17117, 102]** — 6 tokens total

Why [CLS] and [SEP]?

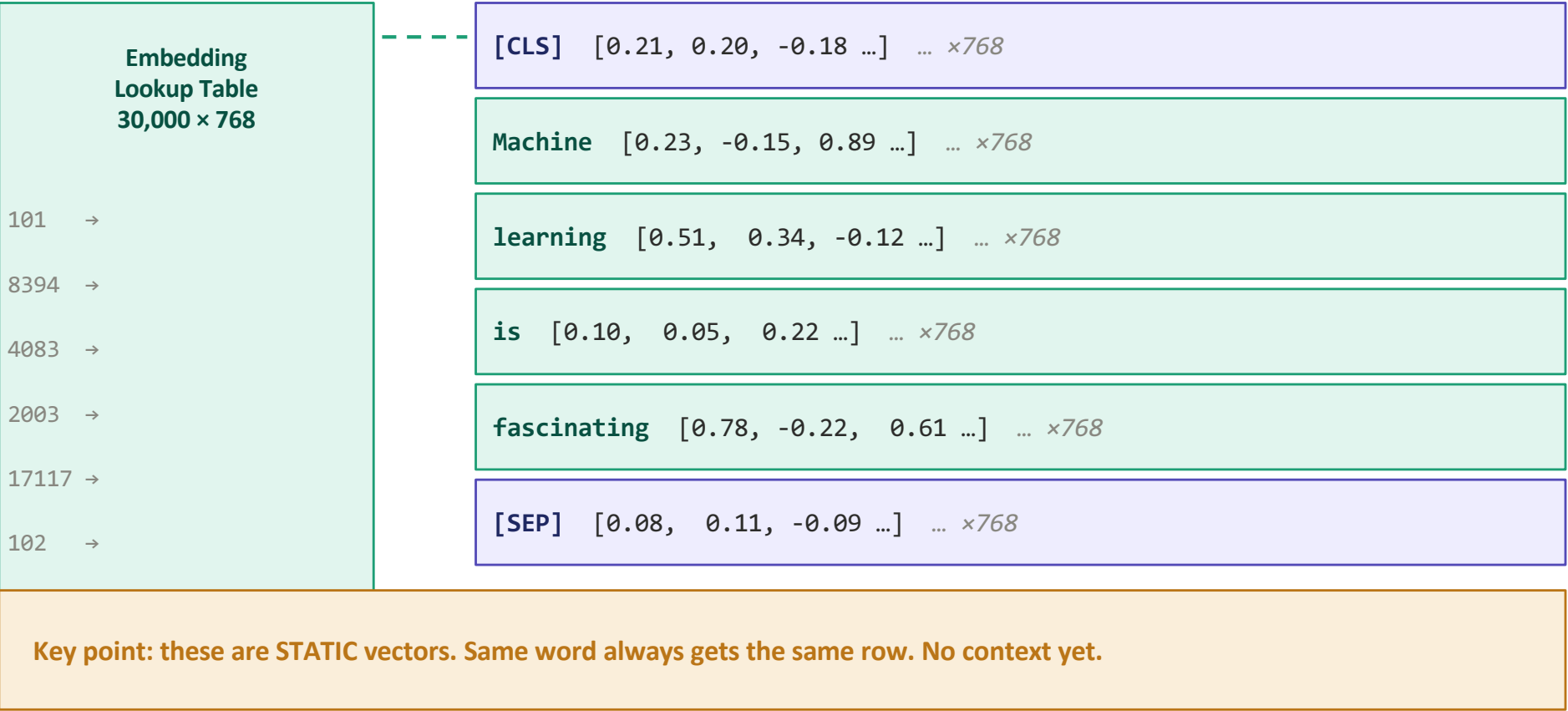
[CLS] (Classification token) — placed at position 0. After all 12 layers, its output vector summarises the entire sentence. Used for sentence-level tasks.

[SEP] (Separator token) — marks the end. Required by BERT's architecture.

BERT Pipeline

Step 2 — Token Embeddings

Each Token ID is looked up in a 30,000 × 768 learned table — one fixed vector per word.



BERT Pipeline



Step 3 — Positional Embeddings Added

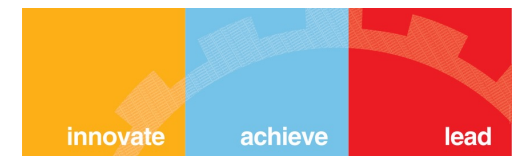
A second 768-dim vector is looked up per position and added elementwise to the token embedding.

$$\text{input_embedding} = \text{token_emb} + \text{pos_emb}$$

Token		token embedding		pos embedding		input embedding
[CLS]	pos 0	[0.21, 0.20...]	+	[0.01, 0.02...]	=	[0.22, 0.22...]
Machine	pos 1	[0.23, -0.15...]	+	[0.02, 0.03...]	=	[0.25, -0.12...]
learning	pos 2	[0.51, 0.34...]	+	[0.03, 0.01...]	=	[0.54, 0.35...]
is	pos 3	[0.10, 0.05...]	+	[0.01, 0.04...]	=	[0.11, 0.09...]
fascinating	pos 4	[0.78, -0.22...]	+	[0.02, -0.01...]	=	[0.80, -0.23...]
[SEP]	pos 5	[0.08, 0.11...]	+	[0.04, 0.02...]	=	[0.12, 0.13...]

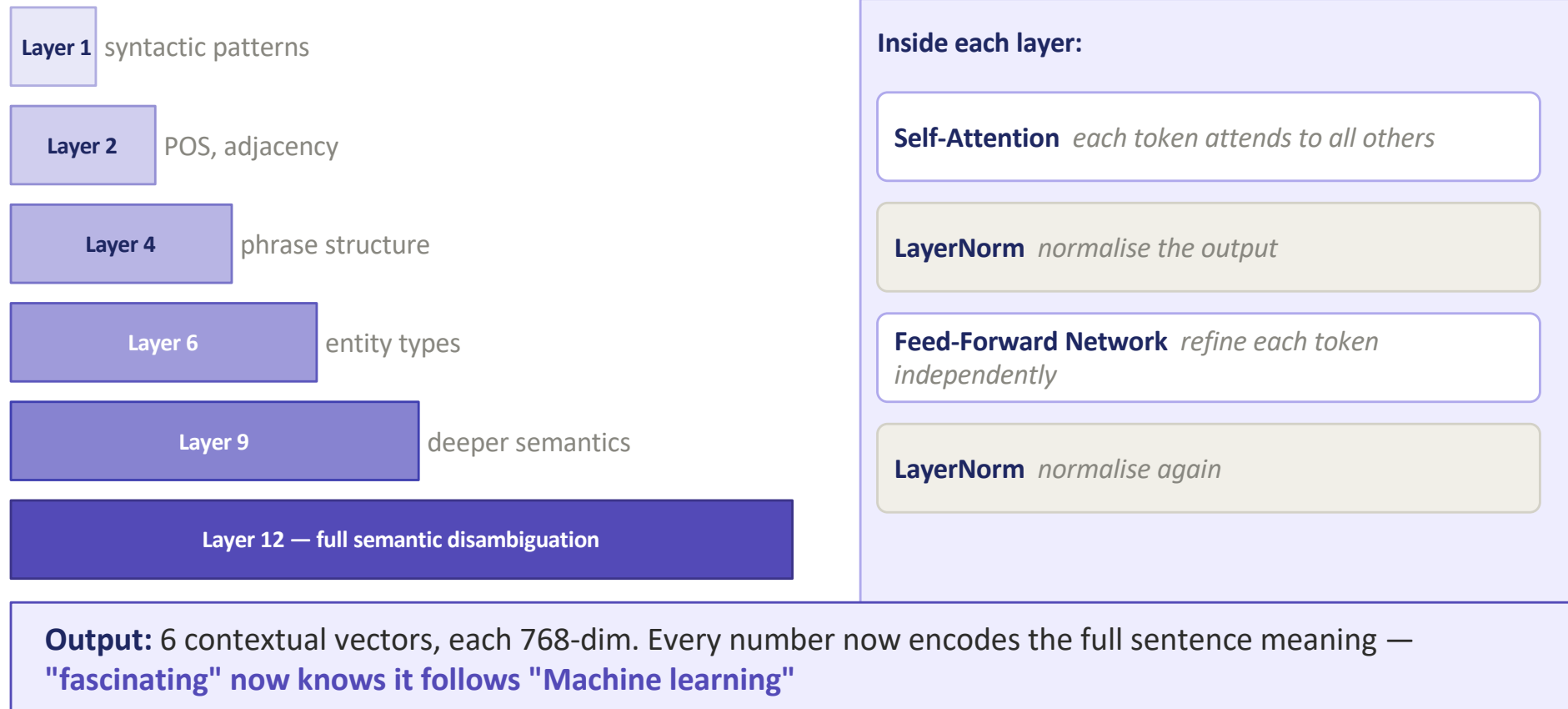
Result: 6 input embeddings — each carries identity + position, ready for the transformer layers.

BERT Pipeline

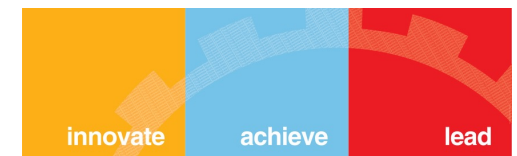


Step 4 — 12 Transformer Layers

Each layer rewrites every token's vector by mixing information from ALL other tokens via self-attention.



BERT Pipeline



Step 5 — Pooling Strategy

We have 6 contextual vectors. To get ONE sentence vector, choose a pooling strategy:

CLS Pooling	Mean Pooling	Max Pooling
$v = h[\text{CLS}]$	$v = (1/n) \sum h_i$	$v_j = \max(h_{1j} \dots h_{6j})$
Take only the [CLS] token's output vector. Default in BERT, RoBERTa, DistilBERT.	Average all 6 token vectors dimension-by-dimension. Used by Sentence-BERT, BGE, GTE.	Take the maximum value per dimension across all token vectors.
Best for: classification, sentiment, NLI	Best for: semantic similarity, search, RAG	Best for: some retrieval models

Final output: **1 vector × 768 numbers** — the full sentence "Machine learning is fascinating" in vector form

How Encoder Transformers Create Embeddings



Self-Attention Mathematics

Query, Key, Value projections:

$$Q = X W_Q, \quad K = X W_K, \quad V = X W_V$$

Attention scores:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Where d_k is dimension of keys (typically 64)

Pooling Strategies

Mean Pooling:

$$v = (1/n) \sum h_i$$

CLS Pooling:

$$v = h[\text{CLS}]$$

Max Pooling:

$$v_j = \max(h_{1j}, h_{2j}, \dots, h_{nj})$$



Key Insight

The final embedding is **contextual** — "bank" gets different embeddings in different sentences because self-attention incorporates surrounding context. This is fundamentally different from word2vec or GloVe (static embeddings).



Embedding Models: Key Players

Open Source Models (Recommended for Production)

Model	Dimensions	Context	Best For	Provider
bge-large-en-v1.5	1,024	512	General purpose, high quality	BAAI
gte-large-en-v1.5	1,024	8,192	Long documents	Alibaba
e5-mistral-7b-instruct	4,096	32,768	Instruction-following, very long context	Microsoft
jina-embeddings-v2	768	8,192	Multilingual, fast inference	Jina AI

Embedding Models: Key Players

Commercial API Models (Highest Quality)

Model	Dimensions	Context	Best For	Cost
text-embedding-3-large	3,072	8,191	Highest quality, production	\$0.13 / 1M tokens
embed-english-v3.0	1,024	512	Compression support, production	\$0.10 / 1M tokens

Selection Guide:

Start with **bge-large-en-v1.5** for general use. Upgrade to commercial APIs if quality justifies cost. Use **gte-large** or **e5-mistral** for long documents (>1000 tokens).

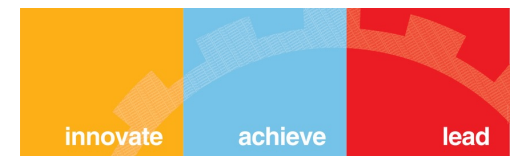
Embedding Models: Complete Reference

Model	Provider	Arch	Layers	Hidden	Output	Training	Context	Type
text-embedding-3-large	OpenAI	Transformer	N/A	N/A	3,072	Contrastive	8,191	API
embed-v3	Cohere	Transformer	N/A	N/A	1,024	Multi-task	512	API
bge-large-en-v1.5	BAAI	BERT	24	1,024	1,024	RetroMAE+Contrastive	512	Open
gte-large-en-v1.5	Alibaba	BERT	24	1,024	1,024	Contrastive	8,192	Open
e5-mistral-7b-instruct	Microsoft	Mistral 7B	32	4,096	4,096	Contrastive pre-training	32,768	Open
jina-embeddings-v2	Jina AI	BERT	12	768	768	Contrastive	8,192	Open



Reference: Complete specifications for implementation decisions. See Slide 15-16 for selection guidance.

Training Objectives:



1. Contrastive Learning

Learning by comparing **similar vs. dissimilar pairs** of inputs to create meaningful embeddings where similar items are closer and dissimilar ones are farther apart.

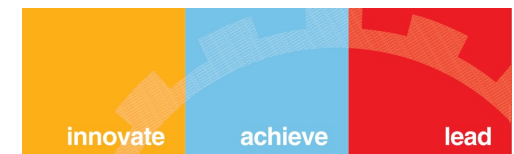
2. Masked Language Modeling (MLM)

A training approach where **random words (tokens) are masked**, and the model learns to **predict the missing words** using surrounding context.

3. RetroMAE (Masked Auto-Encoding)

An advanced masked learning technique where the model **reconstructs masked content with enhanced context understanding**, leading to richer and more robust representations.

Training Objective – Contrastive Learning



Core Objective

Train the model to place similar items close together in embedding space while pushing dissimilar items far apart.

How It Works

The model learns by examining pairs or triplets of examples:

Positive Pair (should be close): "The cat sat on the mat" ↔ "A feline rested on the rug"

Negative Pair (should be far): "The cat sat on the mat" ↔ "How to bake chocolate cookies"

$$\text{Loss} = -\log\left(\frac{\exp(\text{sim}(a, p))}{\sum \exp(\text{sim}(a, n))}\right)$$

where: sim = similarity, a = anchor, p = positive, n = negatives

✓ Advantages

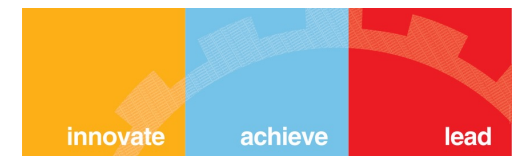
- ✓ Creates semantically meaningful embeddings
- ✓ Works well with large datasets
- ✓ Naturally handles synonyms and paraphrases
- ✓ Self-supervised (doesn't need manual labels)

✗ Challenges

- ✗ Requires careful negative sampling
- ✗ Computationally expensive (many comparisons)
- ✗ Sensitive to batch size
- ✗ Hard negatives mining can be tricky

Models: SimCSE, DPR, SBERT, E5, GTE, mE5, NV-Embed

Training Objective – Masked Language Modeling (MLM)



Core Objective

Predict masked (hidden) words in a sentence using surrounding context. This teaches the model deep language understanding.

How It Works

The model sees sentences with some words replaced by [MASK] tokens and must predict what was hidden.

Original: "The quick brown fox jumps over the lazy dog"

Masked: "The [MASK] brown fox [MASK] over the lazy dog"

Model predicts: [MASK₁] = "quick", [MASK₂] = "jumps"

$$\text{Loss} = -\sum \log P(\text{masked_word} \mid \text{context})$$

Maximize probability of correct word given surrounding words

✓ Advantages

- ✓ Captures bidirectional context
- ✓ No labeled data required
- ✓ Learns rich linguistic patterns
- ✓ Foundation for BERT and similar models

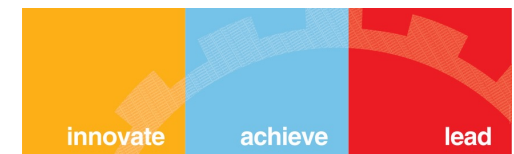
✗ Challenges

- ✗ Slower training than causal LM
- ✗ Pretrain-finetune gap (no [MASK] at inference)
- ✗ Assumes independence of masked tokens
- ✗ Not optimal for generation tasks

Example Models Using MLM: BERT, RoBERTa, SpanBERT, eBERTa]

Training Objective – RetroMAE

(Retrospective Masked Auto-Encoding)



Core Objective

Enhanced masked auto-encoding that reconstructs the entire input from a heavily masked version, forcing the model to learn more compressed and meaningful representations.

How It Works

Unlike standard MLM which masks ~15%, RetroMAE masks 50-70% and uses both encoder and decoder:

Step 1 – Encoder: Process heavily masked input (only 30-50% visible)

Step 2 – Decoder: Reconstruct the ENTIRE original sentence

Example:

Original: "The quick brown fox jumps over the lazy dog"

Masked: "The [M] [M] fox [M] [M] [M] lazy [M]"

Task: Reconstruct all 9 words from just 3 visible ones

Loss = MSE(original_embeddings, reconstructed_embeddings) | With asymmetric encoder-decoder architecture

✓ Advantages

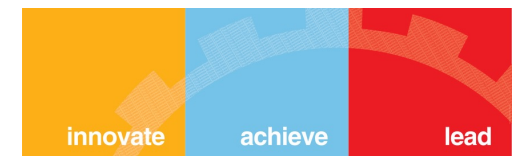
- ✓ Creates highly compressed embeddings
- ✓ Better semantic understanding than standard MLM
- ✓ Efficient for retrieval tasks

✗ Challenges

- ✗ More complex architecture (encoder + decoder)
- ✗ Requires careful tuning of masking ratio
- ✗ Longer training time

Models: RetroMAE, RetroMAE v2, RetroMAE-BEIR

From Embeddings to Search



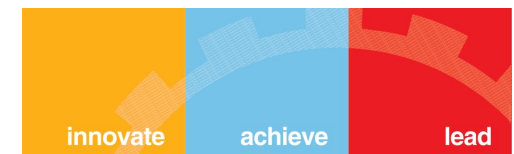
What We've Learned

- ✓ Transformer encoders create rich, contextual embeddings
- ✓ Self-attention captures relationships between words
- ✓ Different pooling strategies extract sentence-level representations

Next: Vector Databases

Learn how to measure similarity, build efficient indexes, and search at scale using approximate nearest neighbor algorithms.

Vector Similarity: Mathematical Foundations



1. Cosine Similarity

$$\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

$$\cos(\theta) = \sum(A_i \times B_i) / (\sqrt{\sum A_i^2} \times \sqrt{\sum B_i^2})$$

Range: $[-1, 1]$

Interpretation:

- 1.0 = identical direction
- 0.0 = orthogonal (unrelated)
- -1.0 = opposite direction

2. Euclidean (L2) Distance

$$d(A, B) = \sqrt{\sum (A_i - B_i)^2}$$

Range: $[0, \infty]$

Interpretation:

- 0 = identical vectors
- Larger = more different
- Sensitive to magnitude

Must normalize vectors first!

3. Dot Product (Inner Product)

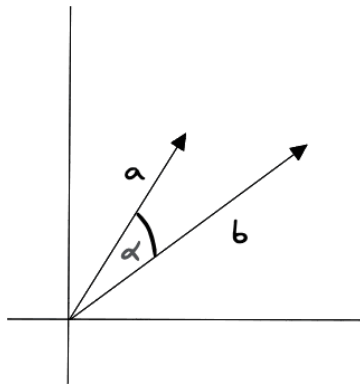
$$A \cdot B = \sum(A_i \times B_i) = \|A\| \|B\| \cos(\theta)$$

Range: $[-\infty, \infty]$ ($[-1, 1]$ if normalized) **Key insight:** If vectors are normalized ($\|A\| = \|B\| = 1$), then dot product = cosine similarity!

Vector Similarity: Visual Intuition

1. Cosine Similarity

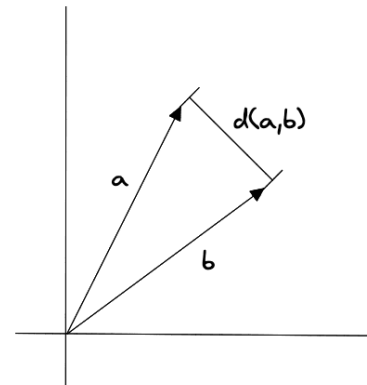
$$\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \|\mathbf{B}\|)$$



Measures angle
between vectors
Range: $[-1, 1]$

2. Euclidean (L2) Distance

$$d(\mathbf{A}, \mathbf{B}) = \sqrt{\sum (\mathbf{A}_i - \mathbf{B}_i)^2}$$



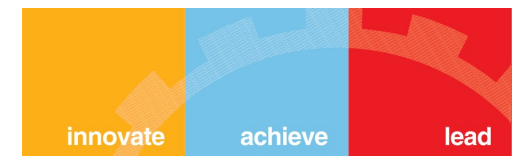
Measures distance in
space
Range: $[0, \infty]$

Feature	Cosine Similarity	Euclidean Distance
Measures	Angle between vectors	Distance between points
Sensitivity	Not sensitive to magnitude	Highly sensitive to magnitude
Range	-1 to 1	0 to ∞
High Value	High similarity	Low similarity
Best For	Text, high-dimensional data, Content-based recommendations prioritizing thematic alignment	Image data, geometry, Product recommendations measuring preference intensity

Code Demo-1

Hands-on implementation of cosine similarity, L2 distance, and dot product

The Computational Challenge



Problem Statement

Given: 10 million document embeddings
(768 dimensions each)

Task: Find top 10 most similar to query
embedding

Computational Complexity

For linear scan:

- Operations per query = $n \times d$
- $n = 10,000,000$ documents
- $d = 768$ dimensions
- Total = 7,680,000,000 multiply-add operations

At 1 billion ops/sec (modern CPU):

Time per query = 7.68 seconds

With 100 queries/sec: Need 768 CPU cores!

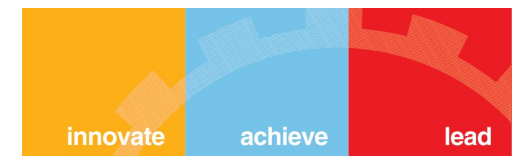
Linear Scan vs ANN Solution

The "Computational Challenge" slide proves that Linear Scan is not a viable solution for vector search at scale.

Linear Scan Failure	ANN Solution
7.68 seconds per query	10 milliseconds per query
Requires 768 CPU cores for 100 QPS	Requires a fraction of the hardware
Brute-force comparison of all 10 million documents	Smart indexing (HNSW) to avoid unnecessary comparisons

The only way to move from the infeasible 7.68 seconds to the real-time 10 milliseconds is by using **Approximate Nearest Neighbor (ANN)** algorithms, which bypass the need for an exact, full scan.

What Is Vector Indexing?



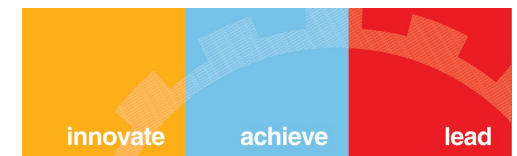
Vector indexing is the process of **structuring your vector data in a way that enables fast similarity search**. Instead of checking every vector one by one, a good index helps narrow down the most promising matches — in milliseconds.

It's like having a super-smart librarian who knows where the right book is **without scanning every shelf**.

This isn't a simple database optimization — it's an **entire field of research**, and one of the reasons vector databases are so powerful.

ANN: Approximate Nearest Neighbor Search

ANN: Approximate Nearest Neighbor Search



The magic behind fast retrieval is a technique called **Approximate Nearest Neighbor (ANN)** search.

With ANN:

- You trade off a tiny bit of **accuracy** for **dramatically faster** search
- Instead of exact neighbors, you get "**close enough**" matches
- Most applications (search, recommendation, RAG, etc.) benefit from this tradeoff

Why? Because semantic vectors already involve approximation — a slight distance difference won't make or break your result.

Approximate Nearest Neighbor (ANN)

Scale Analysis

Documents	Linear Scan	Cost/month
10K	8ms	\$50
100K	77ms	\$50
1M	770ms	\$200
10M	7.7s ✗	\$5,000

With ANN Index (HNSW)

Documents	HNSW	Cost/month
10K	2ms	\$50
100K	3ms	\$50
1M	5ms	\$120
10M	10ms ✓	\$250

The Solution: Approximate Nearest Neighbor (ANN)

Key Insight: We don't need the EXACT top 10 — getting 9/10 correct is often good enough!

ANN algorithms trade tiny accuracy loss (95-99% recall) for 100-1000× speedup by using smart data structures.

1. Graph-Based

Example: HNSW

Build a navigable graph where nodes are vectors, edges connect similar vectors

Pros:

- Excellent recall (>95%)
- Fast queries ($O(\log n)$)
- No training needed

Cons:

- High memory usage
- Slow inserts

2. Partition-Based

Example: IVF

Cluster vectors into partitions, search only relevant partitions

Pros:

- Balanced performance
- Works with compression
- Scalable

Cons:

- Needs training (k-means)
- Sensitive to nprobe

3. Compression-Based

Example: PQ

Compress vectors to reduce memory, search in compressed space

Pros:

- 8-32× less memory
- Billions of vectors

Cons:

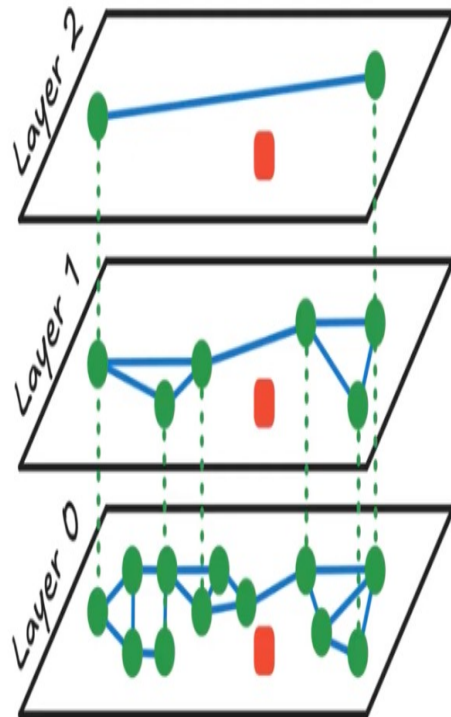
- Lower recall (~90%)
- Needs training

innovate **achieve** **lead**

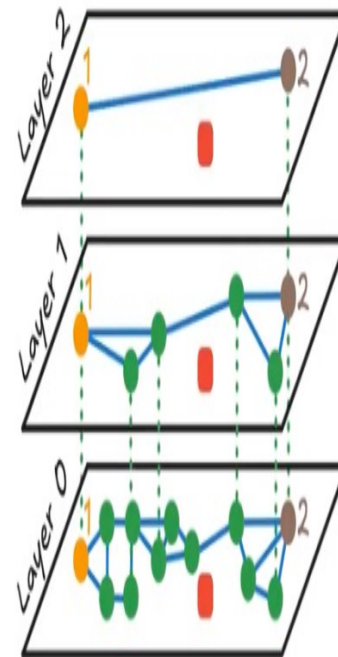
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

HNSW (Hierarchical Navigable Small World)

Core Concept: Multi-Layer Skip List for Vectors

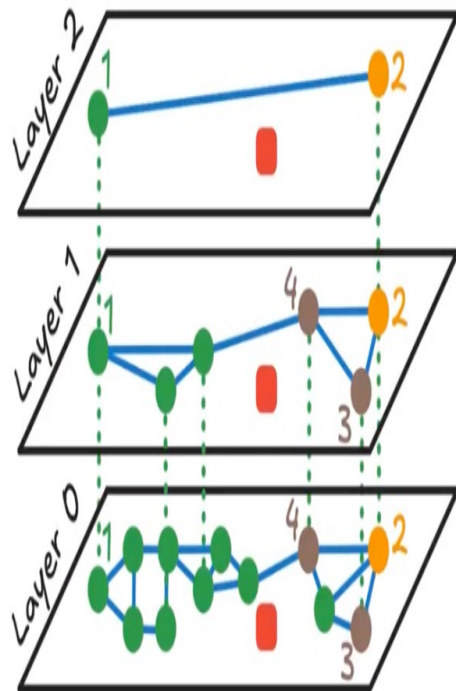


Goal: Find the closest point to the query (red square) without checking all

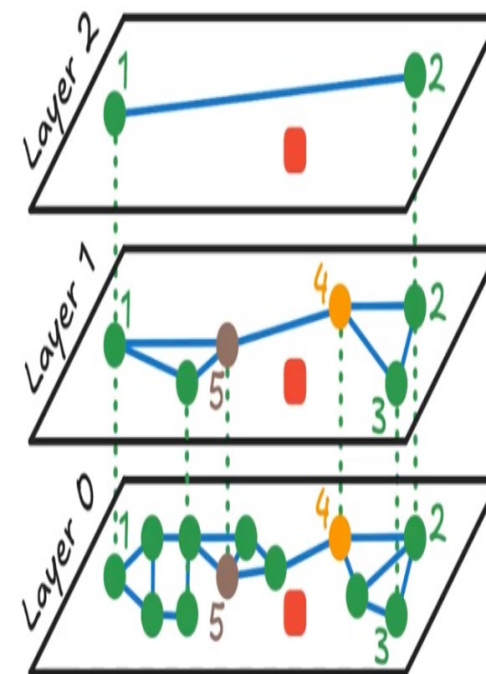


Step 1: The search begins at a random entry point (1) on the highest layer.

HNSW (Hierarchical Navigable Small World)



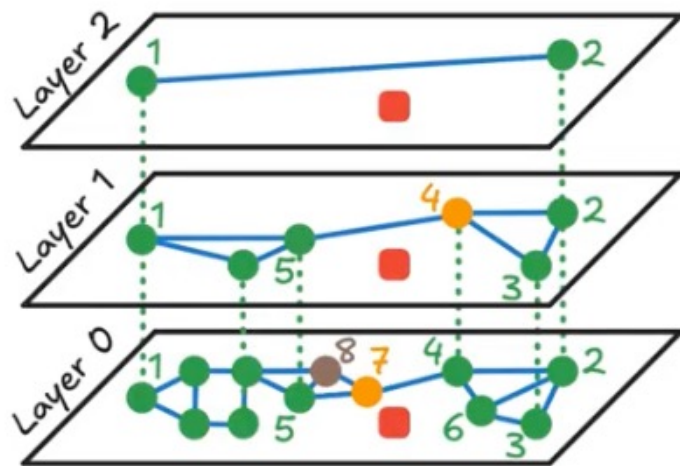
Step 2: After finding the closest point (2), we drop down to Layer 1.



We perform a greedy search in Layer 1, moving to point 4, which is closer.

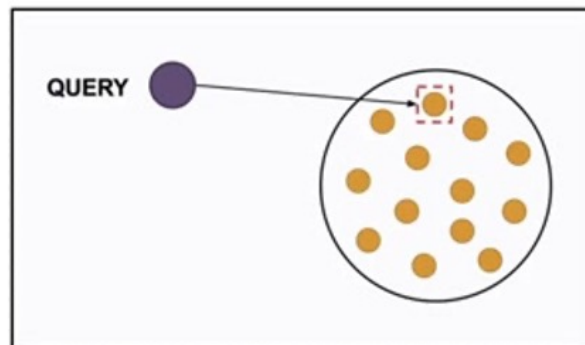
HNSW (Hierarchical Navigable Small World)

Core Concept: Multi-Layer Skip List for Vectors

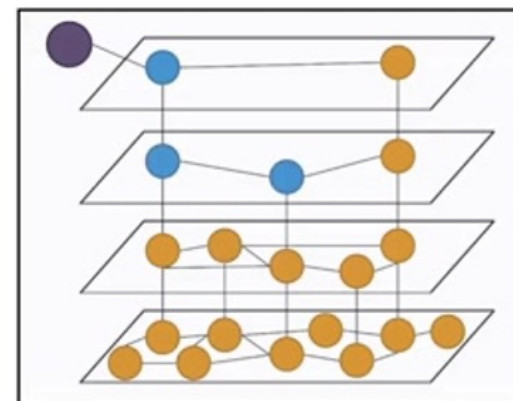


Step 3: On the **ground floor** (Layer 0), a final search reveals point 7 is the **closest**.

Brute-Force: $O(N)$



HNSW: $O(\log N)$



Source: Malkov & Yashunin (2018)

Source: Malkov & Yashunin (2018)

HNSW: Memory Layout



Per Vector Storage Components

Storage Breakdown

- ✓ **Original vector:** $d \times 4$ bytes
- ✓ **Layer info:** 1 byte
- ✓ **Connections:** $M \times 4$ bytes $\times$ layers, (M = max connections per node)

Example (768-dim, $M=16$)

Vector: $768 \times 4 = 3,072$ bytes

Layer: 1 byte

Connections: ~ 64 bytes avg

Total: $\sim 3,136$ bytes per vector

Why M matters so much

M is the single most important parameter in HNSW. It controls three things simultaneously:

- **Search accuracy:** more neighbours = better approximate nearest neighbour recall
- **Search speed:** more neighbours to evaluate per hop = slightly slower traversal
- **Memory:** each extra neighbour costs 4 bytes $\times$ number of layers

HNSW: Memory Calculation by Scale

Scale	Vector Data	Graph Overhead	Total Memory
1M vectors	~3.1 GB (1.02× raw)	~0.5-1 GB (metadata)	~3.6-4.1 GB (1.2-1.3× raw)
10M vectors	~31 GB (1.02× raw)	~10-20 GB (adjacency)	~41-51 GB (1.4-1.7× raw)
100M vectors	~310 GB (1.02× raw)	~200-300 GB (multi-layer)	~510-610 GB (1.7-2.0× raw)

⚠ Production Planning: Memory Budget Guidelines

- **Per-vector data:** Minimal overhead (~1.02×)
- **Graph structure:** 0.4-1.0× additional (depends on M and scale)
- **Practical rule:** Budget 1.5-2.0× raw vector memory for production HNSW

Parameter Tuning: HNSW



M (Connections per Layer)	ef_construction	ef_search (Query Time)
16 – 64 Lower (16): • Less memory • Faster build • Lower recall Higher (64): • More memory • Better recall • Slower inserts Default: M=16 for most cases	100 – 200 Lower (100): • Faster build • Lower quality index Higher (200+): • Slower build • Better index quality • Higher recall Default: ef_construction=200	50 – 500 ef=50: • Fast • ~90% recall ef=100: • Balanced ef=500: • Slow • 99% recall Typical: ef_search=100-200

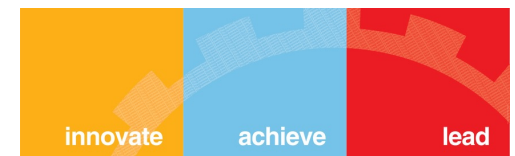
💡 *Pro Tip: Start with defaults (M=16, ef_construction=200, ef_search=100). Profile on your data, then adjust ef_search for latency-recall tradeoff.*

Resources & Further Reading

- **Dense Passage Retrieval** (Karpukhin et al., 2020)
Foundation for modern dense retrieval
- **HNSW** (Malkov & Yashunin, 2018)
Efficient and robust approximate nearest neighbor search
- **Product Quantization** (Jégou et al., 2011)
Vector compression for large-scale search
- **Sentence-BERT** (Reimers & Gurevych, 2019)
Siamese networks for semantic similarity

References

1. https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/readings/cs224n-self-attention-transformers-2023_draft.pdf
2. <https://vizuara.ai>
3. <https://jalammar.github.io/illustrated-transformer/>
4. <https://www.youtube.com/watch?v=fCUy1DZZspM>
5. <https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models>
6. https://bge-model.com/tutorial/1_Embedding/1.2.1.html



Thank you